МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Java сервлеты

Методические указания к лабораторной работе по дисциплине "Объектно-ориентированное программирование"

Ростов-на-Дону 20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

УДК 512.3

Java сервлеты: методические указания – Ростов н/Д: Издательский центр ДГТУ, 20 . – 8 с.

Рассмотрена технология Servlet API, схема создания, установки и использования сервлетов. Даны задания по выполнению лабораторной работы. Методические указания предназначены для студентов направлений 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный редактор:

Издательский центр ДГТУ, 20

Для создания и функционирования web-приложений и корпоративных приложений в Java используются технологии Java EE (Enterprise Edition). В рамках JavaEE прописаны спецификации различных API необходимых для функционирования корпоративных приложений. Одной из таких спецификаций является спецификация Sevlet API. В этой спецификации определены программные компоненты - сервлеты, предназначенные для генерации динамического отклика приложения на внешние запросы. Например, пользователь в браузере задает адрес web-приложения, установленного на каком-то сервере приложений. Для этого он должен знать адрес сервера и имя приложения, кроме того, он может через сам запрос передать приложению какую-нибудь дополнительную информацию. Браузер формирует запрос к серверу и ожидает от него ответа на этот запрос. Ответ, который возвращает сервер браузеру - это чаще всего некоторая html-страница со ссылками на другие ресурсы, прикрепленные к этой странице (например, сценарии JavaScript, изображения, размещенные на странице и т.п.). Ответ сервера отображается в окне браузера и, тем самым, пользователь получает нужную ему информацию. Для формирования ответа серверу нужно выполнить ряд операций. Это могут быть обращения к различным источникам данных, вычисления, фильтрация данных, полученных из разных источников и т.д. Всем этим занимаются различные программные компоненты, установленные на сервере. Такими компонентами, в частности являются сервлеты - программы, установленные на сервере и вызываемые для обслуживания запросов пользователей.

1. Серверы приложений и web-серверы

web- и корпоративные приложения выполняются на сервере

приложений. Сервер приложений должен содержать контейнеры приложений соответствующего типа (например, web- или корпоративных). Контейнер предоставляет выполняющемуся в нем приложению набор сервисов. Приложение, в свою очередь должно предоставить контейнеру информацию о свой структуре, для того, чтобы тот мог управлять этим приложением. Изначально такая информация должна была помещаться в обязательный дескриптор приложения. Дескриптор - это текстовый xml-файл. Для web-приложения дескриптор должен иметь имя web.xml. Для конкретного серавлета в нем указывается имя класса сервлета (с учетом пакета) и то имя, которое будет использоваться внешними запросами для обращения к сервлету. Например,

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<web-app    xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
        http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd"
        version="2.4">
<servlet>
        <servlet-name>lab2</servlet-name>
        <servlet-class>labs.Lab2</servlet-class>
</servlet>
<servlet-mapping>
        <servlet-name>lab2</servlet-name>
        <url-pattern>/Lab2</url-pattern>
</servlet-mapping>

</web-app>
```

Здесь вначале идет стандартный заголовок. Меняться будут только записи в тегах <servlet> и <servlet-mapping>. <servlet-name> задает логическое имя сервлета, необходимое для того, чтобы отличать один установленный на сервере сервлет от другого. <servlet-class> задает имя класса сервлета с учетом пакета, в данном случае сервлет представлен классом Lab2 из пакета labs. <url-pattern> определяет имя, которое нужно использовать в запросах для обращения к сервлету.

Начиная со спецификации 3.0 дескриптор сервлета не является обязательным. Можно использовать аннотации. Если заданы и аннотации, и дескриптор, приоритет будет у дескриптора.

Приложения (вместе со своими дескрипторами, если они используются) размещаются в специальном каталоге сервера приложений. В каком конкретно зависит от используемого сервера. Существует ряд открытых и коммерческих реализаций серверов приложений и упрощенные варианты, являющиеся, например, серверами только web-, но не корпоративных приложений. Необходимо установить какой-то сервер на своем компьютере для выполнения этой и следующих лабораторных работ. В дальнейшем предполагается использование web-сервера Apache Tomcat в конфигурации с дополнительными пакетами - TomEE+.

https://www.apache.org/dyn/closer.cgi/tomee/tomee-8.0.8/apache-tomee-8.0.8-plus.tar.gz

В версии Jakarta EE 9 изменились имена пакетов с javax.* на jakarta

https://www.apache.org/dyn/closer.cgi/tomee/tomee-9.0.0-M7/apache-tomee-9.0.0M7-plus.zip

В примерах ниже предполагается использование пакетов javax.*, то есть Jakarta EE 8 (и, поэтому, TomEE 8).

На сервере Tomcat для web-приложений предусмотрена папка webapps. Чтобы разместить на сервере свое приложение с именем, например, lab2 нужно в папке webapps создать папку lab2 и все, что относится к этому приложению, поместить в эту папку. Для выполнения лабораторных работ можно установить сервер локально, на своем компьютере. Тогда, в случае Tomcat, по умолчанию обращение к серверу происходит через локальный адрес localhost и порт 8080. Например, localhost:8080/lab2/Lab2 - обращение к сервлету Lab2 web-приложения lab2.

2. Сервлеты

Сервлет в программе представляется классом, производным от некоторого стандартного, предопределенного класса.

Стандартный класс javax.servlet.GenericServlet представляет сервлет, обрабатывающий клиентские запросы, которые могут поступать с использованием разных (формально любых) протоколов передачи данных. На практике, сервлеты в основном обрабатывают http-запросы. Такие сервлеты должны быть представлены в программе классами, производными от класса javax.servlet.http.HttpServlet (наследника GenericServlet).

```
package labs;
import javax.servlet.*; import javax.servlet.http.*;
…
public class Lab2 extends HttpServlet {
        …
}
```

GenericServlet и HttpSrvlet дают свои реализации интерфейса javax.servlet.Servlet, который определяет жизненный цикл сервлета.

2.1. Жизненный цикл сервлета

В интерфейсе Servlet объявлены методы

void init(ServletConfig sc) - вызывается при создании объекта класса-сервлета на сервере

void service(ServletRequest req, ServletResponce resp) - вызывается всякий раз, когда нужно обработать запрос и сформировать ответ на него

void destroy() - вызывается перед удалением объекта класса-сервлета.

При инициализации сервлет получает от контейнера ссылку на объект класса, реализующего интерфейс ServletConfig. ServletConfig позволяет сервлету во время его инициализации получить извне значения некоторых параметров (параметров инициализации - Initialization Parameters), которые задаются один раз и далее используются сервлетом все время, пока он функционирует. Эти параметры можно разместить в виде пар имя-значение в дескрипторе приложения - файле web.xml.

```
<web-app>
    <servlet>
        <servlet-name>lab2</servlet-name>
        <servlet-class>labs.Lab2</servlet-class>
        <init-param>
            <param-name>firm</param-name>
            <param-value>MyFirm</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>lab2</servlet-name>
        <url-pattern>/Lab2</url-pattern>
    </servlet-mapping>
```

</web-app>

В интерфейсе ServletConfig объявлены следующие методы:

String getInitParameter(String name) - возвращает значение параметра инициализации с указанным именем, например, в методе init: String firmName = sc. getInitParameter("**firm**");

Enumeration<String> getInitParameterNames() - возвращает перечислимый тип с именами всех параметров инициализации сервлета;

ServletContext getServletContext(); - возвращает ссылку на контекст, в котором выполняется сервлет;

String getServletName(); - возвращает имя сервлета.

Интерфейс ServletContext определяет методы, используемые сервлетом для взаимодействия со своим контейнером. Если приложение содержит несколько сервлетов, то все они находятся в одном контексте и могут обмениваться данными (атрибутами) через этот контекст. Для этого можно использовать методы SevletContext:

Enumeration<String> getAttributeNames();

Object getAttribute (String name);

void setAttribute(String name, Object value);

String removeAttribute(String name)

Если один сервлет добавляет атрибут в контекст (setAttribute), то другие сервлеты из этого же контекста могут получить значение этого атрибута с помощью метода getAttribute. Кроме того, ServletContext позволяет динамически создавать и добавлять в контекст приложения сервлеты, фильтры, слушателей событий (методы createServlet, createFilter, createListener, addServlet, addFilter, addListener см. документацию [1]).

Всякий раз, когда сервер получает запрос, контейнер сервлетов определяет сервлет, который должен сгенерировать ответ и вызывает метод service. При этом соответствующий объект класса-сервлета не создается заново. Контейнер создает и передает методу service объекты классов ServletRequest и ServletResponce, которые представляют для сервлета запрос и ответ соответственно. Через эти объекты сервлет может получать/устанавливать различные параметры, читать входные данные или записывать данные, представляющие ответ на запрос. Например, сревлет может создавать в качестве ответа на запрос html-страницу и отправлять ее в поток вывода, связанный с объектом класса ответа. Класс HttpServlet определяет методы, соответствующие конкретным видам http-запроса и в своем методе service выполняет вызов этих методов. Например, для http-запроса GET предназначен метод void doGet(HttpServletRequest req, HttpServletResponce resp).

```
package labs;
import javax.servlet.*;
import javax.servlet.http.*;

…
public class Lab2 extends HttpServlet {
	@Override
	public void doGet(HttpServletRequest req, HttpServletResponce resp) {
		…
	}
}
```

2.2. Аннотация @WebServlet

Для сервлетов можно не задавать описание в дескрипторе web.xml. Тогда перед объявлением класса-сервлета нужно задать аннотацию @WebServlet. Она объявлена в пакете javax.servlet.annotation.

```
package labs;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
…
	@WebServlet(name="lab2", urlPatterns={"/Lab2"})
	public class Lab2 extends HttpServlet {
		@Override
		public void doGet(HttpServletRequest req, HttpServletResponce resp) {
			…
	}
}
```

Аргумент name аннотации соответствует тегу <servlet-name> в дескрипторе web.xml, а urlPatterns - тегам <url-pattern>. Вместо urlPatterns можно использовать value

```
	@WebServlet(name="lab2", value={"/Lab2"})
	public class Lab2 extends HttpServlet {
		…
	}
```

Если не задать name, будет использовано имя класса. Кроме того, если в аннотации @WebServlet в скобках указано только одно значение, без имени атрибута, то это будет значение атрибута urlPatterns.

```
@WebServlet("/Lab2")
public class Lab2 extends HttpServlet {
        …
}
```

2.3. http-запросы

Протокол HTTP предусматривает несколько видов запросов: GET, POST, HEAD, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATH. Стандартные запросы, предполагающие получение ответа от сервера, а не выполнение служебных операций - это GET и POST запросы.

Например, POST запрос:

Файл start.html

```
<html>
  <title>POST Request</title>
  <body>
    <form method="post" action="/lab2/Lab2">
      <p>
      <table>
      <tbody>
        <tr>
          <td>Value</td>
          <td><input type="text" name="value" size="20"></td>
          <td></td><td><input type="submit" value="Send"></td>
      </tbody>
      </table>
      </p>
    </form>
```

```
  </body>
</html>
```

В случае GET запроса можно так же, как и при POST-запросе использовать форму на стартовой html-странице или выполнить его через адресную строку браузера: http://localhost:8080/**lab2/Lab2?value=12345**

Для html-страницы нужно указать тип http-запроса GET:

```
…
<title>GET Request</title>
  <body>
    <form method="get" action="/lab2/Lab2">
…
```

В браузере выполняем запрос на загрузку стартовой страницы start.html

http://localhost:8080/lab2/start.html

Если имя html-страницы **index.html** то она будет загружаться по умолчанию и указывать ее имя необязательно, запрос будет выглядеть так:

http://localhost:8080/lab2/

Получить значения параметров, переданные через http-запрос в сервлете можно с помощью метода String getParameter(String name) класса HttpServletRequest. Ответ сервлет формирует с помощью класса Writer, ссылку на объект которого дает метод PrintWriter getWriter() throws IOException класса HttpServletResponse.

Например, если выполнен GET-запрос

http://localhost:8080/lab2/Lab2?**value**=12345

и сервлет должен сформировать в качестве ответа html-страницу, содержащую текст Value=12345

```
public void doGet(HttpServletRequest req, HttpServletResponce resp) {
```

```
        String data = req.getParameter(value);
        resp.setContentType("text/plain");
        try(java.io.PrintWriter out = resp.getWriter();) {
                out.println("Value=" + data);
        } catch(java.io.IOException ioe) {
                ioe.printStackTrace();
        }
}
```

Можно не использовать браузер, а написать свою клиентскую Java-программу, которая будет выполнять GET или POST запросы и получать ответ, сформированный сервлетом. Для этого нужно задать адрес, по которому выполняется запрос (объект класса java.net.URL), установить соединение с сервером для работы по http-протоколу (класс HttpURLConnection), использовать методы этого класса для задания параметров соединения. Кроме того, обычно нужно передать в запросе некоторые данные

```
        // GET-запрос
import java.io.*;
import java.net.*;
import java.net.http.*;
        …
        URL url = null;
        HttpURLConnection con = null;      try {
                url = new URL("http://localhost:8080/lab2/Lab2?value=12345");
                con = (HttpURLConnection)url.openConnection();
                con.setRequestMethod("GET");
                con.setRequestProperty("Content-Type", "text/html");
```

```
        con.setConnectTimeout(5000);
        con.setReadTimeout(5000);
 } catch(Exception e) {
        …
 }
 finally {
        if(con != null) con.close();
}

 // POST-запрос
 …
 import java.nio.charset.StandardCharsets;
 …
 try {
        url = new URL("http://localhost:8080/lab2/Lab2");
        con = (HttpURLConnection)url.openConnection();
        con.setRequestMethod("POST");
        con.setRequestProperty("User-Agent", "Java client");
        con.setRequestProperty(
                "Content-Type", "application/x-www-form-urlencoded");
        con.setConnectTimeout(5000);
        con.setReadTimeout(5000);
        String args = " value=12345";
        byte[] data = args.getBytes(StandardCharsets.UTF_8);
        con.setDoOutput(true);
        try (var wr = new DataOutputStream(con.getOutputStream())) {
                wr.write(data);
```

```
        }
    } catch(Exception e) {
        …
    }
    finally {
        …
    }
```

Получить ответ от сервлета (по запросу любого типа) можно следующим образом

```
    …
    int status = 0;
    StringBuffer data = new StringBuffer(); // ответ в виде текста
    BufferedReader in = null;
    try {
        status = con.getResponseCode();
        if(status > 299) { // Error
            …
            return;
        }
    in = new BufferedReader(new InputStreamReader(con.getInputStream()));
        String str = null;
        while((str=in.readLine()) != null) data.append(str);
    } catch(Exception e) {
        …
    }
    finally {
```

```
        in.close();
        con.disconnect();
    }
    …
```

3. Развертывание сервлетов

Чтобы сервлет можно было использовать, его нужно не только откомпилировать, но и правильно установить (развернуть) на сервере приложений, содержащем контейнер сервлетов.

Компиляция:

javac -cp .;"C:\Program Files\Apache Software Foundation\Tomcat 9.0\lib\servlet-api.jar" -d . Lab2.java

IntelliJ Idea Community не имеет встроенной поддержки проектов Java EE, поэтому в ней можно создавать обычный Java проект с добавленной библиотекой servlet-api.jar из папки lib сервера Tomcat (TomEE).

Развертывание:

Найти каталог, в который установлен Tomcat (или TomEE), например, C:\Program Files\Apache Software Foundation\Tomcat 9.0, перейти в каталог webapps. Создать папку приложения, например, lab2. В ней необходимо создать папку WEB-INF, в которой, в свою очередь нужно создать папку classes. Здесь, с учетом пакета, должны располагаться откомпилированные классы сервлета. Если развертывание выполняется с помощью дескриптора (файл web.xml), то он должен располагаться в папке WEB-INF. Если используется стартовая html-страница (например, index.html), ее нужно расположить в каталоге lab2. Здесь же должны располагаться и другие ресурсы, используемые приложением (серверные страницы Java, сценарии JavaScript, изображения, мультимедийные ресурсы). В итоге должно получиться следующее:

../Tomcat 9.0/webapps/**lab2** - здесь файл **index.html**

|-**WEB-INF** - здесь дескриптор **web.xml**

|-**classes**

|-**labs** (имя пакета)

|-**Lab2.class**

Ресурсы необходимые сервлету располагают обычно в подкаталогах корневой папки приложения (lab2), например, изображения - в подкаталоге lab2/images. Внешние библиотеки (jar-файлы) необходимые приложению помещают в подкаталог lib каталога WEB-INF.

../lab2

|-**images** - здесь изображения

|-WEB-INF

|-classes

|-labs

|-Lab2.class

|-**lib** - внешние библиотеки

Создавать такую систему каталогов на сервере для каждого устанавливаемого приложения неудобно. Стандартный подход заключается в упаковке всей системы каталогов в файл архива с расширением war (от Web ARchive). Для примера выше, это должен быть архив lab2.war. Его следует разместить в каталоге webapps сервера Tomcat (TomEE) без распаковки (Tomcat сам распакует его).

../Tomcat 9.0/webapps/**lab2.war**

Создается war-файл как обычный java-архив с помощью утилиты jar:

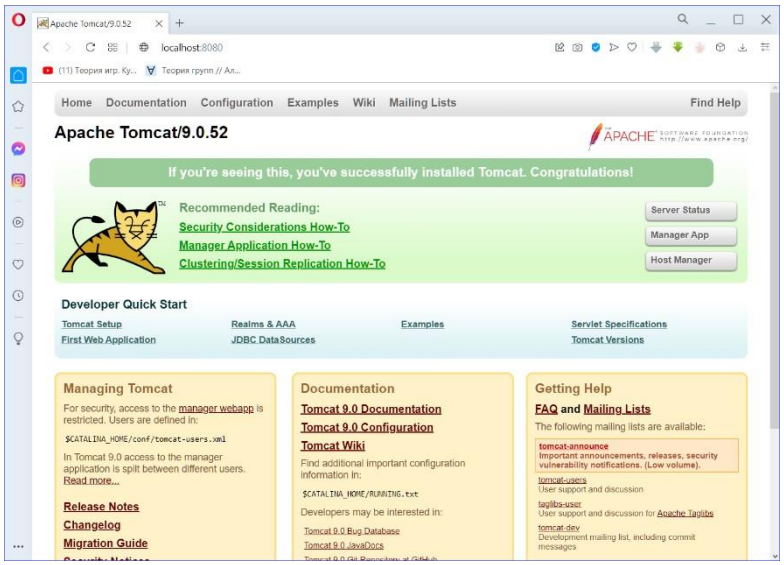
В командной строке, текущий каталог ../Tomcat 9.0/webapps/

jar -**cf** lab2.war WEB-INF index.html

создается (ключ **c**) архив - файл (ключ **f**) lab2.war, включающий в себя все содержимое каталога WEB-INF (с подкаталогами) и файл index.html, кроме того, архив будет также содержать каталог META-INF с файлом MANIFEST.MF.

Архив lab2.war должен быть расположен в каталоге webapps сервера Tomcat.

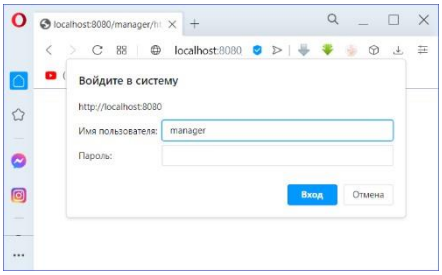
Устанавливать, запускать, останавливать, удалять web-приложения, которыми управляет Tomcat можно также с помощью Tomcat Manager.



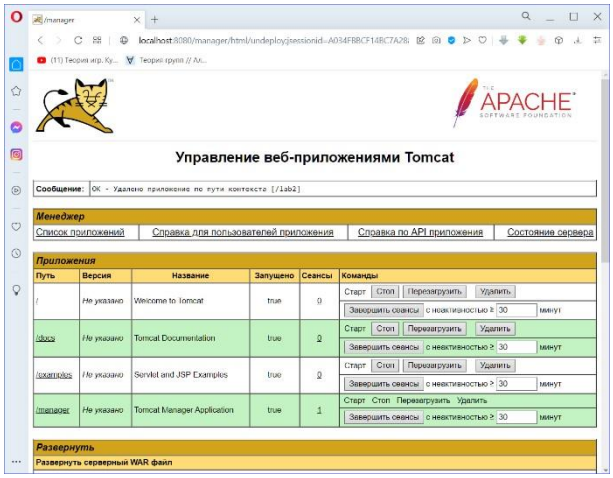
Запустить Tomcat Manager можно, как и любую другую установленную в системе программу, или с домашней страницы Tomcat (правый верхний угол окна, кнопка Manager App). Чтобы использовать Manager нужно быть зарегистрированным пользователем, причем с предопределенной ролью manager-gui. По умолчанию в Tomcat нет зарегистрированных пользователей. Для добавления пользователя нужно отредактировать файл tomcat-users.xml, расположенный в каталоге C:\Program Files\Apache Software Foundation\Tomcat 9.0\conf. В тег <tomcat-users> нужно добавить вложенный тег

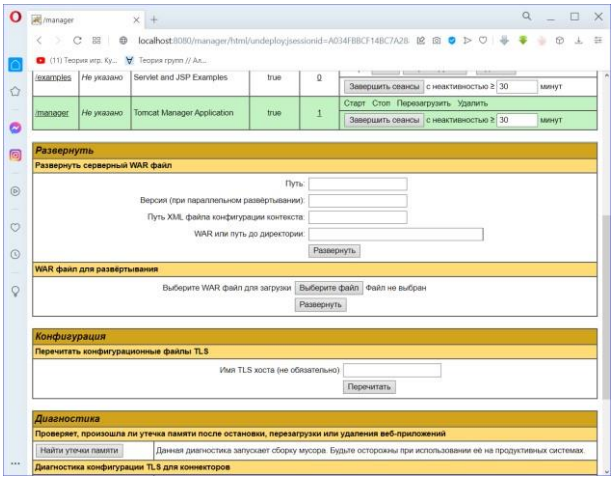
<user username="**manager**" password="**pass**" roles="**manager-gui**"/>

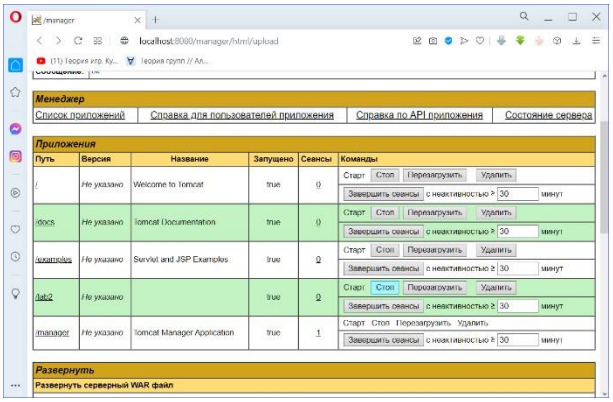
Имя пользователя и пароль произвольные, более того, пароль необязателен <user username="manager" password="" roles="manager-gui"/> но роль должна быть как минимум одна - manager-gui.



Теперь можно войти в Manager с заданной учетной информацией. Отображается список установленных web-приложений. Ниже - раздел **WAR файл для развертывания**. Кнопка "Выберите файл" позволяет выбрать файл (например, lab2.war). Кнопка "Развернуть" - развернуть приложение на сервере.







Приложение разворачивается, содержимое war-файла распаковывается в папку с именем, совпадающим с именем архива (например, lab2) и сервлет запускается на сервере. Это можно видеть в окне Tomcat Manager (появилось приложение lab2). Можно его остановить, перегрузить, удалить (удаляются и папка lab2 и файл lab2.war).

4. Сессии

HTTP-протокол предполагает, что каждый запрос обрабатывается независимо от предыдущих запросов (не сохраняется состояние). В реальных задачах клиент посылает несколько запросов связанных друг с другом

некоторой логикой и последовательностью выполнения. Например, в интернет-магазине покупка предполагает запросы на нужные товары, работу с корзиной покупок. Все это связанные запросы, относящиеся к одному и тому же клиенту. Нужно связать эти запросы в отдельную группу, отделив от других запросов, сделанных другими клиентами. Для этого организуется сессия. У каждой сессии есть свой идентификатор. Этот идентификатор сохраняется с использованием разных схем, например, файлов куки, если они доступны, и передается от сервера к клиенту и наоборот при каждом ответе и запросе. Сессию в программе представляет объект класса HttpSession. Ссылку на этот объект в сервлете можно получить через объект, представляющий запрос - HttpServletRequest. Метод String getId() класса HttpSession позволяет получить идентификатор сессии. При первом запросе от клиента на сервере создается уникальный идентификатор. Метод boolean isNew() позволяет узнать, создан ли идентификатор только сейчас или это уже очередной запрос в данной сессии. long getCreationTime() и long getLastAccesedTime() возвращают соответственно время создания и время последнего обращения к сессии в миллисекундах от 1 января 1970 года. Чтобы получить из этого значения дату и время в привычном виде можно использовать класс Date. В сессию можно добавлять атрибуты в виде ключ-значение с помощью метода setAttribute(key, value) и получать добавленные атрибуты Object getAttribute(key).

```
public void doGet(HttpServletRequest req, HttpServletResponce resp) {
        HttpSession session = req.getSession();
        PrintWriter out = resp.getWriter();
        out.println("Session id: " + session.getId());
        out.println("Creation time: " + new Data( session.getCreationTime() ));
        out.println("Last accessed time: " + new Data(
        session.getLastAccessedTime() ));
```

```
        if( session.isNew() ) session.setAttribute("name", "Mr.X");
        else out.println(session.getAttribute("name"));
}
```

Если запросы выполняются не из браузера, а из приложения, автоматической пересылки содержимого куки-файлов не происходит и отслеживание сессии нужно выполнять явно.

5. Фильтры

Спецификация Servlet API (начиная с версии 2.3) позволяет использовать программные единицы, которые могут вызываться до того, как запрос будет передан сервлету или после того, как запрос будет обработан сервлетом - фильтры. Фильтры могут образовывать цепочки, в которых данные сгенерированные одним фильтром поступают на вход другого фильтра. Фильтры могут выполнять некоторые общие преобразования, такие как аутентификация, перекодирование запроса из одной кодировки символов в другую, сжатие и распаковку передаваемой информации.

Фильтром является класс, реализующий интерфейс javax.servlet.Filter. Жизненный цикл фильтра, заданный в Filter, определяют три метода

void init(FilterConfig fcfg) throws ServletException,

void destroy() и

void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain) throws ServletException, IOException.

6. Переадресация запросов и перенаправление отклика

Переадресация запроса - это передача запроса другому сервлету своего приложения, перенаправление отклика - обращение к другому сервлету

другого приложения, развернутого на данном (или другом) сервере для генерации ответа на запрос.

Если в некотором web-приложении есть два сервлета, текущий <url-pattern>/servlet1<url-pattern> и другой, для которого <url-pattern>/servlet2<urlpattern>. Тогда в методе doGet (или doPost) servlet1 можно выполнить переадресацию на servlet2 следующим образом

```
public void doGet(HttpServletRequest req, HttpServletResponce resp) {
        …
        req.getRequestDispatcher("servlet2").forward(req, resp);
}
```

Перенаправление отклика можно выполнить с помощью метода HttpServletResponse.sendRedirect() (см. документацию [1]).

7. Обработка xml-файлов

xml - это промышленный стандарт, позволяющий создавать языки разметки гипертекстовых документов. Примером такого языка является HTML 5. xml-совместимые (или просто xml-документы), это гипертекстовые документы, в которые помимо содержимого помещается еще и информация о разметке документа. В случае html-документа разметка позволяет указать, например, что некоторый текст должен начинаться с нового параграфа, должен выводиться в указанном цвете и т.п. Многие информационные системы используют xml-документы для обмена информацией. Например, центральный банк России предоставляет стандартный бесплатный (с ограничением по количеству запросов) API получения информации о курсах валют и драгоценных металлов. Клиент посылает http-запрос и получает в ответ xml-файл с котировками. Примеры запросов можно посмотреть по ссылке https://cbr.ru/development/SXML/?0D13AC55-EFB7-1C48-82AF-

4C4A299E05FF_kis_cup_C6FA3ED5_6D17_47D1_B6E2_F4B02CC905E0_

Для получения текущего курса разных валют по отношению к рублю на заданную дату, скажем, двадцатое августа 2021 года (20/08/2021) нужно выполнить запрос

http://www.cbr.ru/scripts/XML_daily.asp?date_req=20/08/2021

Получить коды валют можно по запросу https://cbr.ru/scripts/XML_val.asp?d=0

Чтобы получить динамику котировок драгоценных металлов за указанный период времени нужно выполнить запрос

http://www.cbr.ru/scripts/xml_metall.asp?date_req1=02/07/2021&date_req2=01/08/2021

Коды драгоценных металлов:

1 - золото

2 - серебро

3 - платина

4 - палладий

Чтобы получить динамику изменения курса некоторой валюты (например, доллара, код R01235) по отношению к рублю за некоторый промежуток времени (например, с 02/03/2021 по 01/08/2021) нужно выполнить запрос

http://www.cbr.ru/scripts/XML_dynamic.asp?date_req1=02/03/2021&date_req2=01/ 08/2021&VAL_NM_RQ=R01235

Будет получен xml-документ со следующим содержимым


```
<?xml version="1.0" encoding="windows-1251"?>
<ValCurs ID="R01235" DateRange1="02.03.2021" DateRange2="01.08.2021" name="Foreign Currency Market Dynamic">
 <Record Date="02.03.2021" Id="R01235">
        <Nominal>1</Nominal>
        <Value>74,0448</Value>
```

```
</Record>
 <Record Date="03.03.2021" Id="R01235">
        <Nominal>1</Nominal>
        <Value>74,5755</Value>
</Record>
 <Record Date="04.03.2021" Id="R01235">
        <Nominal>1</Nominal>
        <Value>73,5187</Value>
</Record>
…
</ValCurs>
```

Здесь тэги <ValCurs>, </ValCurs> и содержимое между ними определяют отдельный (корневой) элемент документа. <Record>, </Record> вложенные элементы. Data и Id - это атрибуты тэга Record, а "02.03.2021" и "R01235" конкретные значения этих атрибутов. Тэги <Nominal>, </Nominal> и <Value>, </Value> - определяют соответствующие элементы, вложенные в Record. У них нет атрибутов, но есть содержимое, например, 1 и 74,0448, соответственно. Для обработки xml-документов в Java можно использовать разные подходы, низкоуровневые (DOM, SAX) и более высокоуровневые как JAXB. В примере ниже используется DOM (не требует использования дополнительных модулей, поддержка встроена в Java SDK). Такой подход используется при работе с небольшими документами. Весь xml-документ считывается и на его основе строится дерево узлов, представляющих содержимое документа. Узел в программе представляется объектом класса org.w3c.dom.Node, последовательность узлов - списком org.w3c.dom.NodeList. Для работы с атрибутами используется класс org.w3c.dom.NamedNodeMap, в котором можно хранить пары имя атрибута - значение атрибута. Документ в целом

представляется классом org.w3c.dom.Document, отдельный элемент классом org.w3c.dom.Element. Класс javax.xml.parsers.DocumentBuilderFactory позволяет создать объект класса javax.xml.parsers.DocumentBuilder который предоставляет метод parse для создания объекта класса Document, содержащего узлы построенного на основе xml-документа дерева.

```
import java.util.Scanner;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document; import org.w3c.dom.Element;
import org.w3c.dom.NamedNodeMap;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
…
DocumentBuilderFactory docfactory = null;
 DocumentBuilder docbuilder = null;  Document doc = null;
 try {
        docfactory = DocumentBuilderFactory.newInstance();
        docbuilder = docfactory.newDocumentBuilder();
        File f = new File("currency.xml");
        doc = docbuilder.parse(f); /* есть перегруженный метод parse,
получающий ссылку на InputStream */
} catch(Exception e) {
        …
        e.printStackTrace();
}
doc.getDocumentElement().normalize();
```

```
NodeList nodes = doc.getElementsByTagName("Record"); /* список всех элементов Record */
…
Node node = nodes.get(i); // получаем i-й элемент Record из списка
NamedNodeMap map = node.getAttributes();
Node dateNode = map.getNamedItem("Date");
Node idNode = map.getNamedItem("Id");
if(node.getNodeType() == Node.ELEMENT_NODE) {
	NodeList nl = node.getChildNodes(); /* вложенные элементы для Record */
	…
	String name = nl.item(j).getNodeName(); /* название вложенного элемента Nominal или Value */
	String data = nl.item(j).getTextContent(); /* значение тэга Nominal или Value */
	…
}
…
```

ЗАДАНИЕ 1. Создайте простейшую систему авторизации/регистрации: html-страницу с формой для ввода логина и пароля и кнопкой Submit, для отправки на сервер GET-запроса с информацией из формы, сервлет, обрабатывающий http-запросы и возвращающий в случае успешной авторизации/регистрации соответствующее сообщение и текущие дату/время (сервера), если логин/пароль не правильный, возвращается сообщение и номер попытки. После трех неудачных попыток авторизация в текущей сессии становится невозможной и об

этом выводится сообщение. Учетная информация пользователей (логин и хэш пароля) хранятся на сервере в xml-файле.

ЗАДАНИЕ 2. Создайте сервлет, обрабатывающий GET-запросы и возвращающий в ответ html-страницу, содержащую в зависимости от значений переданных параметров, следующую информацию: идентификатор сессии и время ее создания, текущее время на сервере, количество запросов к сервлету в текущей сессии. Используйте браузер (создайте нужную html-страницу) для взаимодействия с сервлетом.

ЗАДАНИЕ 3. Модифицируйте, если необходимо, сервлет из задания 1 так, чтобы он мог обрабатывать также и POST-запросы. Используйте JavaFX для создания клиентского приложения, генерирующего httpзапросы как в задании 1 и отображающего в своем окне ответ полученный от сервлета. Пользователь должен иметь возможность выбирать тип http-запроса (GET или POST).

ЗАДАНИЕ 4. Создайте сервлет, по запросам пользователей предоставляющий информацию о котировках валют или драгоценных металлов центрального банка России на текущую дату или за указанный период, от одной даты до другой. Результат возвращается в виде htmlстраницы. Пользователь из браузера отправляет запрос и в браузере видит результаты запроса (для этого нужно создать html-страницу с нужной формой).

ЗАДАНИЕ 5. Создайте JavaFX приложение, посылающее запросы сервлету из задания 3. Ответ сервлета отображается в окне приложения

в виде таблицы и, если запрос сдержит диапазон дат, также в виде графика, представляющего динамику изменения выбранной валюты за указанный период.

Литература

1. Jakarta EE. Jakarta Servlet Specification. Version 5.0. Jakarta Servlet Team, https://projects.eclipse.org/projects/ee4j.servlet. – 2020, 177 p.
2. В.В.Смелов "Оновы web-программирования на Java" : учеб.-метод. пособие. – Минск: БГТУ. – 2009, 140 с.
3. Б.Перри "Java сервлеты и JSP: сборник рецептов", 3-е рус. изд. – М.:КУДИЦПресс. – 2009, 768 с.
4. Д.Хеффельфингер "Java EE 6 и сервер приложений GlassFish 3". – М.:ДМК-Пресс. – 2013, 416 с.
5. J.Murach, M.Urban "Murach's Java Servlets and JSP", 3-d ed. – Mike Murach & Associates Inc. – 2014, 767c.
6. J.Juneau "Java EE 8 Recipes", 2-d ed. – Apress. – 2018, 792c.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.